

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyze	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

*I **Shaik Ayesha Tabassum(192525074)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: S.Ayesha Tabassum

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate;	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations

	correct final answer			
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.

ANS:

```
mysql> CREATE TABLE Department (  
->   Dept_ID INT PRIMARY KEY,  
->   Dept_Name VARCHAR(30)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE Employee (  
->   Emp_ID INT PRIMARY KEY,  
->   Emp_Name VARCHAR(30),  
->   Salary INT,  
->   Dept_ID INT,  
->   FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
-> );  
Query OK, 0 rows affected (0.08 sec)  
  
mysql> INSERT INTO Department VALUES  
-> (101,'HR'),  
-> (102,'Sales'),  
-> (103,'IT'),  
-> (104,'Finance');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO Employee VALUES  
-> (1,'Ayesha',45000,101),  
-> (2,'Rahul',60000,103),  
-> (3,'John',50000,102),  
-> (4,'Sara',70000,103),  
-> (5,'David',55000,104);  
Query OK, 5 rows affected (0.01 sec)  
Records: 5 Duplicates: 0 Warnings: 0
```

```
mysql> select * from Department;  
+-----+-----+  
| Dept_ID | Dept_Name |  
+-----+-----+  
| 101 | HR |  
| 102 | Sales |  
| 103 | IT |  
| 104 | Finance |  
+-----+-----+  
4 rows in set (0.00 sec)  
  
mysql> select * from Employee;  
+-----+-----+-----+-----+  
| Emp_ID | Emp_Name | Salary | Dept_ID |  
+-----+-----+-----+-----+  
| 1 | Ayesha | 45000 | 101 |  
| 2 | Rahul | 60000 | 103 |  
| 3 | John | 50000 | 102 |  
| 4 | Sara | 70000 | 103 |  
| 5 | David | 55000 | 104 |  
+-----+-----+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql> SELECT Emp_Name, Dept_Name  
-> FROM Employee  
-> INNER JOIN Department  
-> ON Employee.Dept_ID = Department.Dept_ID;  
+-----+-----+  
| Emp_Name | Dept_Name |  
+-----+-----+  
| Ayesha | HR |  
| John | Sales |  
| Rahul | IT |  
+-----+-----+
```

```
+-----+-----+  
| Emp_Name | Dept_Name |  
+-----+-----+  
| Ayesha | HR |  
| John | Sales |  
| Rahul | IT |  
| Sara | IT |  
| David | Finance |  
+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql> SELECT Emp_Name  
-> FROM Employee  
-> INNER JOIN Department  
-> ON Employee.Dept_ID = Department.Dept_ID  
-> WHERE Dept_Name='IT';  
+-----+  
| Emp_Name |  
+-----+  
| Rahul |  
| Sara |  
+-----+  
2 rows in set (0.00 sec)  
  
mysql> SELECT Emp_Name, Salary, Dept_Name  
-> FROM Employee  
-> INNER JOIN Department  
-> ON Employee.Dept_ID=Department.Dept_ID;  
+-----+-----+-----+  
| Emp_Name | Salary | Dept_Name |  
+-----+-----+-----+  
| Ayesha | 45000 | HR |  
| John | 50000 | Sales |  
+-----+-----+-----+
```

```

+-----+-----+-----+
| Emp_Name | Salary | Dept_Name |
+-----+-----+-----+
| Ayesha   | 45000  | HR        |
| John     | 50000  | Sales     |
| Rahul    | 60000  | IT        |
| Sara     | 70000  | IT        |
| David    | 55000  | Finance   |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT Emp_Name, Salary
-> FROM Employee
-> WHERE Salary =
-> (
-> SELECT MAX(Salary)
-> FROM Employee
-> );
+-----+-----+
| Emp_Name | Salary |
+-----+-----+
| Sara     | 70000  |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT Emp_Name, Salary
-> FROM Employee
-> WHERE Salary >
-> (
-> SELECT AVG(Salary)
-> FROM Employee
-> );
+-----+-----+
| Emp_Name | Salary |
+-----+-----+
| Rahul    | 60000  |
| Sara     | 70000  |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT Emp_Name
-> FROM Employee
-> WHERE Dept_ID =
-> (
-> SELECT Dept_ID
-> FROM Department
-> WHERE Dept_Name='HR'
-> );
+-----+
| Emp_Name |
+-----+
| Ayesha   |
+-----+
1 row in set (0.00 sec)

```

Logic used:

The SQL query retrieves the required information by using **joins** and **subqueries**. A **JOIN** combines data from two or more tables using a common column (such as Dept_ID) to display related information like employee names and their department names. A **subquery** is a query written inside another query, where the **inner query executes first** to calculate or retrieve a value (such as the maximum salary, average salary, or department ID). The **outer query then uses this result** to filter and display the required records. This approach helps retrieve accurate and meaningful data efficiently from the database.

2. Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.

ANS:

Relations

EMPLOYEE(Emp_ID, Emp_Name, Salary, Dept_ID)

Emp_ID	Emp_Name	Salary	Dept_ID
1	Ayesha	45000	101
2	Rahul	60000	103
3	John	50000	102
4	Sara	70000	103
5	David	55000	104

DEPARTMENT(Dept_ID, Dept_Name)

Dept_ID	Dept_Name
101	HR
102	Sales
103	IT
104	Finance

Query 1: Display Employee Name and Department Name

Relational Algebra

π Emp_Name, Dept_Name (Employee $\bowtie$ Department)

Logic Used

- Join the **Employee** and **Department** relations using the common attribute Dept_ID.
- Display only the employee name and department name using the projection (π) operator.

Output: Ayesha–HR, Rahul–IT, John–Sales, Sara–IT, David–Finance.

Query 2: Display Employees Working in the IT Department

Relational Algebra

π Emp_Name (σ Dept_Name='IT' (Employee $\bowtie$ Department))

Logic Used

- Join the Employee and Department relations.
- Select (σ) only the records where the department is IT.
- Project (π) only the employee names.

Output: Rahul, Sara.

Query 3: Display Employees with Salary Greater Than ₹50,000

Relational Algebra

π Emp_Name, Salary (σ Salary > 50000 (Employee))

Logic Used

- Select employees whose salary is greater than ₹50,000.
- Display only the employee name and salary.

Output: Rahul, Sara, David.

Query 4: Display Employees with Salary Greater Than ₹50,000 Working in IT

Relational Algebra

π Emp_Name (σ Salary > 50000 AND Dept_Name = 'IT' (Employee $\bowtie$ Department))

Logic Used

- Join the Employee and Department relations.
- Apply two conditions: salary greater than ₹50,000 and department equal to IT.
- Display only the employee names.

Output: Rahul, Sara.

Query 5: Display Employees Working in HR or Finance

Relational Algebra

π Emp_Name (σ Dept_Name = 'HR' OR Dept_Name = 'Finance' (Employee $\bowtie$ Department))

Logic Used

- Join the Employee and Department relations.
- Select employees belonging to the HR or Finance departments.
- Display only the employee names.

Output: Ayesha, David.

Conclusion

Relational algebra retrieves data by using operators such as **Selection (σ)** to filter rows, **Projection (π)** to select required columns, and **Join ($\bowtie$)** to combine related relations using a common attribute like Dept_ID. Complex queries are solved by combining these operators with logical conditions such as **AND** and **OR**, allowing only the required records to be retrieved efficiently. This provides a clear mathematical foundation for database query processing.

- Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.

ANS:

A Student Management System stores information about students, departments, and courses. Each student belongs to one department and can enroll in multiple courses.

```
mysql> use student;
Database changed
mysql> CREATE TABLE Student (
  ->   Student_ID INT PRIMARY KEY,
  ->   Student_Name VARCHAR(30),
  ->   Department VARCHAR(20),
  ->   Age INT
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Student VALUES
  -> (101,'Ayesha','CSE',20),
  -> (102,'Rahul','ECE',21),
  -> (103,'John','CSE',19);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Age |
+-----+-----+-----+-----+
| 101 | Ayesha | CSE | 20 |
| 102 | Rahul | ECE | 21 |
| 103 | John | CSE | 19 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Student
  -> WHERE Department='CSE';
+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Age |
+-----+-----+-----+-----+
| 101 | Ayesha | CSE | 20 |
| 103 | John | CSE | 19 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> UPDATE Student
  -> SET Age = 21
  -> WHERE Student_ID = 103;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> DELETE FROM Student
  -> WHERE Student_ID = 102;
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Student
  -> VALUES (104,'Sara','IT',20);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM Student;
+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Age |
+-----+-----+-----+-----+
| 101 | Ayesha | CSE | 20 |
| 103 | John | CSE | 21 |
| 104 | Sara | IT | 20 |
+-----+-----+-----+-----+
```

Conclusion:

The relational schema organizes student data into a structured table with a **primary key** (Student_ID) to uniquely identify each student. SQL retrieval queries use **SELECT** to fetch required records, while update queries such as **INSERT**, **UPDATE**, and **DELETE** are used to add, modify, and remove records. This enables efficient storage, retrieval, and maintenance of data in the database.

4. Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.

ANS:

```
mysql> SELECT Emp_Name
-> FROM Employee
-> WHERE Salary >
-> (
-> SELECT AVG(Salary)
-> FROM Employee
-> );
+-----+
| Emp_Name |
+-----+
| Rahul    |
| Sara     |
+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT AVG(Salary)
-> FROM Employee;
+-----+
| AVG(Salary) |
+-----+
| 56000.0000   |
+-----+
1 row in set (0.00 sec)
```

```
mysql> SELECT Emp_Name
-> FROM Employee
-> WHERE Salary > 56000;
+-----+
| Emp_Name |
+-----+
| Rahul    |
+-----+
```

```
mysql> SELECT Emp_Name
-> FROM Employee
-> WHERE Salary > 56000;
+-----+
| Emp_Name |
+-----+
| Rahul    |
| Sara     |
+-----+
2 rows in set (0.00 sec)
```

Step 1: Identify the Table

First, consider the **Employee** table, which contains employee details such as Employee ID, Employee Name, Salary, and Department ID. The query will use this table to retrieve the required information.

Step 2: Understand the SQL Query

The given SQL query contains a **nested subquery**, which means one query is written inside another query. The inner query is executed first, and its result is used by the outer query to produce the final output.

Step 3: Execute the Inner Query

The database first executes the inner query:

```
SELECT AVG(Salary)
FROM Employee;
```

This query calculates the average salary of all employees. By adding all the salaries and dividing by the total number of employees, the average salary is found to be **₹56,000**. This value is then passed to the outer query.

Step 4: Execute the Outer Query

After receiving the average salary (₹56,000), the outer query compares each employee's salary with this value. It checks whether the salary of each employee is greater than the average salary

Step 5: Evaluate the Condition

Each employee record is examined one by one. Employees whose salary is greater than ₹56,000 satisfy the condition, while the others are ignored. In this example, Rahul earns ₹60,000 and Sara earns ₹70,000, so they satisfy the condition.

Step 6: Display the Output

Finally, the database displays the names of the employees who satisfy the condition. Therefore, the output of the query is **Rahul** and **Sara**.

Conclusion

The query demonstrates how a **nested subquery** works. The **inner query executes first** to calculate the average salary, and the **outer query uses that result** to filter and retrieve the required records. This method is useful when the main query depends on a value calculated by another query.

5. Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency.

ANS:

```
mysql> use employee;
Database changed
mysql> SELECT Emp_Name, Dept_Name
-> FROM Employee
-> INNER JOIN Department
-> ON Employee.Dept_ID = Department.Dept_ID;
+-----+-----+
| Emp_Name | Dept_Name |
+-----+-----+
| Ayesha   | IT        |
| Rahul    | HR        |
| Priya    | Finance   |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT Emp_Name,
-> (
-> SELECT Dept_Name
-> FROM Department
-> WHERE Department.Dept_ID = Employee.Dept_ID
-> ) AS Dept_Name
-> FROM Employee;
+-----+-----+
| Emp_Name | Dept_Name |
+-----+-----+
| Ayesha   | IT        |
| Rahul    | HR        |
| Priya    | Finance   |
+-----+-----+
3 rows in set (0.00 sec)
```

Comparison

JOIN	Subquery
Combines two or more tables directly.	Executes one query inside another query.
Faster and more efficient for related tables.	Slightly slower because the subquery may execute repeatedly.
Easy to retrieve multiple columns from different tables.	Best for calculations and filtering (MAX, MIN, AVG).
Preferred for large databases.	Suitable for simple lookups and nested conditions.

Evaluation

- **JOIN** is generally **more efficient** because it combines tables in a single operation and performs better with large datasets.
- **Subqueries** are useful when the result of one query is required by another query, especially for calculations or filtering.
- For retrieving related data from multiple tables, **JOIN is the recommended approach** because it is faster, easier to read, and more efficient.

Advantages and Disadvantages of JOIN

Advantages

- Faster and more efficient for large datasets.
- Combines data from multiple tables in a single query.
- Easy to retrieve related information.
- Improves query performance.

Disadvantages

- Queries become complex when many tables are joined.
- Incorrect join conditions may produce duplicate or incorrect results.
- Difficult to understand for beginners.

Advantages and Disadvantages of Subquery

Advantages

- Easy to write and understand for simple problems.
- Useful for calculations such as MAX(), MIN(), AVG(), and COUNT().
- Helps break complex queries into smaller parts.
- Suitable when one query depends on the result of another.

Disadvantages

- Slower than joins for large datasets.
- May execute the inner query multiple times, reducing performance.
- Complex nested subqueries are difficult to maintain and debug.
- Not suitable for retrieving large amounts of related data.

Conclusion

Both **JOINS** and **Subqueries** can solve the same problem. However, **JOINS** are generally preferred because they provide better performance, reduce execution time, and are more suitable for retrieving related data from multiple tables. Subqueries are mainly used when one query depends on the result of another query.

6. Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.

ANS:

```
mysql> CREATE VIEW Employee_View AS
-> SELECT Emp_ID, Emp_Name, Dept_ID
-> FROM Employee;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Employee_View;
+-----+-----+-----+
| Emp_ID | Emp_Name | Dept_ID |
+-----+-----+-----+
| 1      | Ayesha   | 101     |
| 2      | Rahul    | 103     |
| 3      | John     | 102     |
| 4      | Sara     | 103     |
| 5      | David    | 104     |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

Step 1: Design the Database Table

First, create an **Employee** table containing employee details such as Employee ID, Employee Name, Salary, and Department ID. This table stores both general and confidential information. Since the salary is sensitive data, it should not be accessible to all users.

Step 2: Create the View

Next, create a **view** by selecting only the required columns from the Employee table. In this example, the view includes Employee ID, Employee Name, and Department ID while excluding the Salary column. This creates a virtual table that hides confidential information from users.

```
CREATE VIEW Employee_View AS
SELECT Emp_ID, Emp_Name, Dept_ID
FROM Employee;
```

Step 3: Retrieve Data Using the View

After creating the view, users can retrieve data by querying the view instead of the original Employee table. The view displays only the selected columns, ensuring that confidential information such as salary is not visible.

```
SELECT * FROM Employee_View;
```

Step 4: Provide Restricted Data Access

The view restricts users from accessing sensitive information by displaying only the required data. Even though the original Employee table contains salary details, users who access the view can see only the employee ID, employee name, and department ID. This ensures controlled access to the database.

Step 5: Justify the Use of Views for Security

Views improve database security by hiding confidential data such as salaries, passwords, or personal information. They allow administrators to control which data each user can access, reducing the risk of unauthorized access and protecting sensitive information.

Step 6: Justify the Use of Views for Abstraction

Views provide data abstraction by hiding the complexity of the original database tables. Users interact with a simple virtual table instead of the complete database structure. This makes queries easier to understand and use without exposing the internal design of the database.

Conclusion

Views are virtual tables that provide a secure and simplified way to access database information. They improve **security** by restricting access to sensitive data and provide **abstraction** by hiding unnecessary details from users. Therefore, views make database management more secure, efficient, and user-friendly.

7. Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

ANS:

```
mysql> CREATE TABLE Student (
  -> Student_ID INT PRIMARY KEY,
  -> Student_Name VARCHAR(30),
  -> Department VARCHAR(20),
  -> Marks INT
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Student VALUES
  -> (101, 'Ayesha', 'CSE', 85),
  -> (102, 'Rahul', 'ECE', 70),
  -> (103, 'John', 'CSE', 90),
  -> (104, 'Sara', 'EEE', 65),
  -> (105, 'David', 'ECE', 80),
  -> (106, 'Priya', 'CSE', 95);
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql> select * from student;
```

Student_ID	Student_Name	Department	Marks
101	Ayesha	CSE	85
102	Rahul	ECE	70
103	John	CSE	90
104	Sara	EEE	65
105	David	ECE	80
106	Priya	CSE	95

6 rows in set (0.00 sec)

```
mysql> SELECT Student_Name, Dept_Name, Marks
  -> FROM Student
  -> INNER JOIN Department
  -> ON Student.Dept_ID = Department.Dept_ID;
```

Student_Name	Dept_Name	Marks
Ayesha	CSE	85
Rahul	ECE	70
John	IT	90
Sara	EEE	65

4 rows in set (0.00 sec)

```
mysql>
mysql> SELECT Student_Name
  -> FROM Student
  -> WHERE Marks > 80;
```

Student_Name
Ayesha
John

2 rows in set (0.00 sec)

```
mysql> SELECT s.Student_Name, d.Dept_Name
  -> FROM Student s
  -> INNER JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Student_Name	Dept_Name
Ayesha	CSE
Rahul	ECE
John	IT
Sara	EEE

4 rows in set (0.00 sec)

Step 1: Create the Database

First, create a database named **Department** and select it using the USE command. This provides a workspace to store and manage the required tables.

Step 2: Create the Department Table

Next, create the **Department** table with two columns: **DeptID** and **DeptName**. Here, **DeptID** is defined as the **Primary Key** to uniquely identify each department.

Step 3: Create the Student Table

Create the **Student** table with the attributes **StudentID**, **StudentName**, **DeptID**, and **Marks**. The **DeptID** is declared as a **Foreign Key** that references the **Department** table, establishing a relationship between the two tables.

Step 4: Insert Sample Data

Insert the department details such as **CSE**, **ECE**, and **EEE** into the Department table. Then insert the student records with their names, department IDs, and marks into the Student table. This provides the data required for query execution.

Step 5: Write the Relational Algebra Expression

Write the relational algebra expression to retrieve the **Student Name** and **Department Name**.

Relational Algebra:

π StudentName, DeptName (Student $\bowtie$ Department)

This expression uses the **Join ($\bowtie$)** operator to combine the Student and Department tables using **DeptID**. The **Projection (π)** operator displays only the required attributes: **StudentName** and **DeptName**.

Step 6: Convert the Relational Algebra into SQL

Convert the relational algebra expression into an equivalent SQL query using **INNER JOIN**.

```
SELECT StudentName, DeptName
FROM Student
INNER JOIN Department
ON Student.DeptID = Department.DeptID;
```

The SQL query joins both tables using **DeptID** and retrieves the student name along with the corresponding department name.

Step 7: Display the Output

The query displays the names of all students with their respective departments.

StudentName	DeptName
Ayesha	CSE
Roops	ECE
Pranathi	CSE
Abhighna	EEE

Step 8: Analyze the Differences

Relational Algebra uses mathematical operators such as **Projection (π)** and **Join ($\bowtie$)** to represent database operations, whereas SQL uses simple English keywords such as **SELECT**, **FROM**, and **INNER JOIN**. Relational Algebra is mainly used for the theoretical representation of queries, while SQL is the practical language used to execute queries in a Database Management System. Both produce the same result, but SQL is easier to understand and implement.

Conclusion

The relational algebra expression and the equivalent SQL query retrieve the same information. Relational Algebra provides the theoretical foundation for database queries, while SQL is the implementation language used to access and manipulate data efficiently in real-world database systems.

8. Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.

ANS:

```
mysql> UPDATE department SET Dept_Name = 'CSE' WHERE Dept_ID = 101;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1 Changed: 0 Warnings: 0

mysql> UPDATE department SET Dept_Name = 'ECE' WHERE Dept_ID = 102;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> UPDATE department SET Dept_Name = 'IT' WHERE Dept_ID = 103;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> UPDATE department SET Dept_Name = 'EEE' WHERE Dept_ID = 104;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> select * from department;
+-----+-----+
| Dept_ID | Dept_Name |
+-----+-----+
| 101 | CSE |
| 102 | ECE |
| 103 | IT |
| 104 | EEE |
+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT Student_Name, Dept_Name
-> FROM Student
-> INNER JOIN Department
-> ON Student.Dept_ID = Department.Dept_ID;
+-----+-----+
| Student_Name | Dept_Name |
+-----+-----+
| Ayesha | CSE |
| Rahul | ECE |
| John | IT |
| Sara | EEE |
+-----+-----+
4 rows in set (0.00 sec)

mysql> CREATE INDEX idx_dept
-> ON Student(Dept_ID);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE VIEW Student_Department AS
-> SELECT Student_Name, Dept_Name
-> FROM Student
-> INNER JOIN Department
-> ON Student.Dept_ID = Department.Dept_ID;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Student_Department;
+-----+-----+
| Student_Name | Dept_Name |
+-----+-----+
| Ayesha | CSE |
| Rahul | ECE |
| John | IT |
+-----+-----+
```

Performance Evaluation

- **Subquery:** Slower because it executes repeatedly for each student.
- **JOIN:** Faster because it combines both tables in one operation.
- **INDEX:** Improves search and join speed.
- **VIEW:** Simplifies repeated queries and improves usability.

Conclusion

Using **JOIN**, **INDEX**, and **VIEW** optimizes SQL queries by reducing execution time, improving performance, and making database operations more efficient. This is the preferred approach for large databases.

9. Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.

ANS:

```
mysql> CREATE TABLE Student (  
-> Student_ID INT PRIMARY KEY,  
-> Student_Name VARCHAR(30),  
-> Department VARCHAR(20),  
-> Marks INT  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO Student VALUES  
-> (101, 'Ayesha', 'CSE', 85),  
-> (102, 'Rahul', 'ECE', 70),  
-> (103, 'John', 'CSE', 90),  
-> (104, 'Sara', 'EEE', 65),  
-> (105, 'David', 'ECE', 80),  
-> (106, 'Priya', 'CSE', 95);  
Query OK, 6 rows affected (0.01 sec)  
Records: 6 Duplicates: 0 Warnings: 0  
  
mysql> select * from student;  
+-----+-----+-----+-----+  
| Student_ID | Student_Name | Department | Marks |  
+-----+-----+-----+-----+  
| 101 | Ayesha | CSE | 85 |  
| 102 | Rahul | ECE | 70 |  
| 103 | John | CSE | 90 |  
| 104 | Sara | EEE | 65 |  
| 105 | David | ECE | 80 |  
| 106 | Priya | CSE | 95 |  
+-----+-----+-----+-----+  
6 rows in set (0.00 sec)
```

```
mysql> SELECT Department, AVG(Marks) AS Avg_Marks  
-> FROM Student  
-> GROUP BY Department  
-> HAVING AVG(Marks) >  
-> (  
-> SELECT AVG(Marks)  
-> FROM Student  
-> );  
+-----+-----+  
| Department | Avg_Marks |  
+-----+-----+  
| CSE | 90.0000 |  
+-----+-----+  
1 row in set (0.00 sec)  
  
mysql> SELECT AVG(Marks)  
-> FROM Student;  
+-----+  
| AVG(Marks) |  
+-----+  
| 80.8333 |  
+-----+  
1 row in set (0.00 sec)
```

Logic Used

- GROUP BY groups students according to their department.
- AVG() calculates the average marks of each department.
- The **nested query** calculates the overall average marks of all students.
- HAVING compares each department's average with the overall average.
- Only departments with an average greater than the overall average are displayed.

Conclusion

This query combines **grouping (GROUP BY)**, **aggregation (AVG())**, and a **nested subquery** to solve a real-world problem. It helps identify departments that perform better than the overall average, making it useful for academic performance analysis in a student management system.

10. Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.

ANS:

```
mysql> SELECT Student_Name, Dept_Name, Marks
-> FROM Student
-> INNER JOIN Department
-> ON Student.Dept_ID = Department.Dept_ID;
+-----+-----+-----+
| Student_Name | Dept_Name | Marks |
+-----+-----+-----+
| Ayesha       | CSE       | 85    |
| Rahul        | ECE       | 76    |
| John         | IT        | 90    |
| Sara         | EEE       | 65    |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT Student_Name, Marks,
-> (
-> SELECT Dept_Name
-> FROM Department
-> WHERE Department.Dept_ID = Student.Dept_ID
-> ) AS Dept_Name
-> FROM Student;
+-----+-----+-----+
| Student_Name | Marks | Dept_Name |
+-----+-----+-----+
| Ayesha       | 85    | CSE       |
| Rahul        | 76    | ECE       |
| John         | 90    | IT        |
| Sara         | 65    | EEE       |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> CREATE VIEW Student_Details AS
-> SELECT Student_Name, Dept_Name, Marks
-> FROM Student
-> INNER JOIN Department
-> ON Student.Dept_ID = Department.Dept_ID;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SELECT * FROM Student_Details;
+-----+-----+-----+
| Student_Name | Dept_Name | Marks |
+-----+-----+-----+
| Ayesha       | CSE       | 85    |
| Rahul        | ECE       | 76    |
| John         | IT        | 90    |
| Sara         | EEE       | 65    |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

Justification

- **JOIN:** Best choice for retrieving related data from multiple tables because it is faster and more efficient.
- **Subquery:** Suitable when one query depends on the result of another, but it is generally slower for large datasets.
- **View:** Best for repeated queries, improving security and simplifying data access.

Conclusion

The best approach for this requirement is to use an **INNER JOIN**, as it efficiently combines data from multiple relations in a single query. **Subqueries** are preferred for dependent calculations, while **views** are useful for simplifying repeated queries and providing secure, restricted access to database information.